

Assignment (1)

Part 1 (Data Visualization)

(1) Since the Data Frame contains records from multiple years, Create a subset of the data to one specific year, say 2016, using the. query().

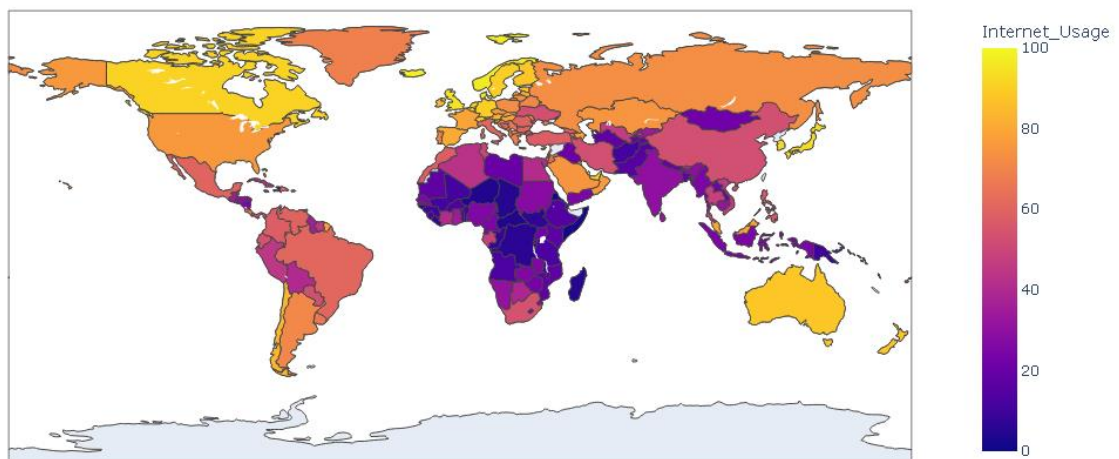
To perform a precise analysis for a specific time period, the dataset was filtered using the .query() method to extract records exclusively for the year 2016.

```
internet_2016 = internet_usage_df.query("Year == 2016")
```

(2) Generate an interactive worldwide choropleth map using the subset data from part (1) and the choropleth function of the plotly library. The following parameters to be set:

- **locations:** "Code"
- **color:** "Individuals using the Internet (% of population)"
- **hover_name:** "Country"
- **color_continuous_scale:** px.colors.sequential.Plasma.

Internet Usage Across the World (% Population) - 2016



This section analyzes global internet usage using a choropleth map created with Plotly Express. The dataset represents the percentage of individuals using the Internet across countries over multiple years.

Key Observations (2016)

- Western regions such as North America and Western Europe exhibit very high internet penetration rates.
- Canada and Australia demonstrate internet usage levels comparable to, and in some cases higher than, many European countries.
- Several developing regions, particularly parts of Africa and South Asia, show significantly lower internet adoption rates.

Global Trend Over Time

- Internet penetration has increased steadily worldwide over the years.
- Developed countries reached high saturation levels earlier due to advanced digital infrastructure.
- Developing countries experienced rapid growth, especially after 2005.
- Although the global digital divide is narrowing, noticeable disparities still exist between developed and developing regions.

(3) What can you say about the internet usage in the western world compared to the east?

The visualization reveals a pronounced "**Digital Divide**" between the Western world and many parts of the East.

- **The Western World:** Countries in North America and Western Europe exhibit very high internet usage rates, often exceeding **80–90%** of the population. This is attributed to advanced technological infrastructure and strong economic development.
- **The East:** The pattern is highly diverse; while developed Eastern economies like Japan and South Korea show penetration rates comparable to the West, many countries in South Asia and Africa demonstrate significantly lower adoption rates.

Conclusion:

The western world has significantly higher internet penetration compared to many eastern and developing regions. This reflects stronger digital infrastructure, higher income levels, and earlier technological adoption.

(4) How the percentage of individual using internet in Canada and Australia is compared to most European countries?

Based on the 2016 choropleth map, Canada and Australia exhibit internet usage rates that are highly comparable to those of most Western European nations.

All these regions fall within the highest tier of digital penetration, with usage rates typically exceeding 80–90% of the population.

They are comparable to top European nations such as Germany, France, and the UK.

Conclusion:

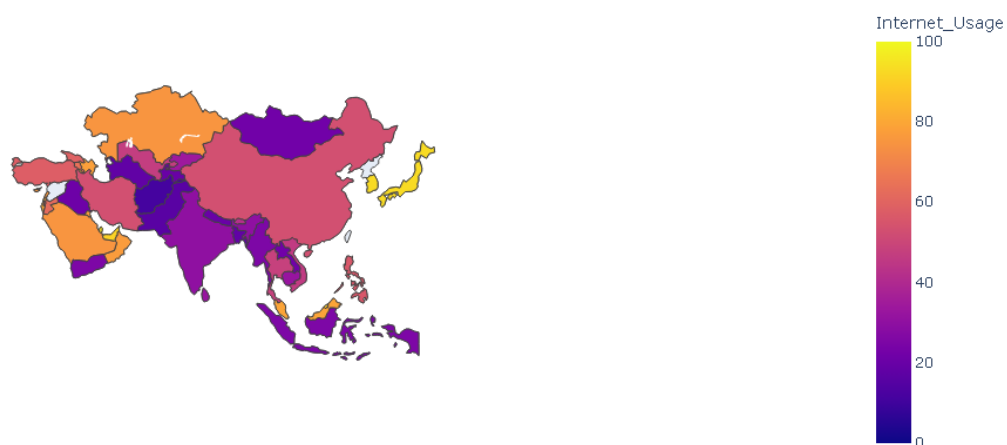
Canada and Australia are among the most digitally connected countries globally and are on par with leading European countries.

(5) Add title text to the choropleth map setting the title_text parameter, that is, `fig.update_layout(title_text='Internet usage across the world (% population) -2016')`

Done

(6) Suppose we are only interested in seeing internet usage across the continent of Asia, we can do this setting geo_scope parameter of the `fig.update_layout` to any of the continent, e.g. north america | south america | africa | asia | europe | usa.

Internet Usage Across Asia (% Population) - 2016

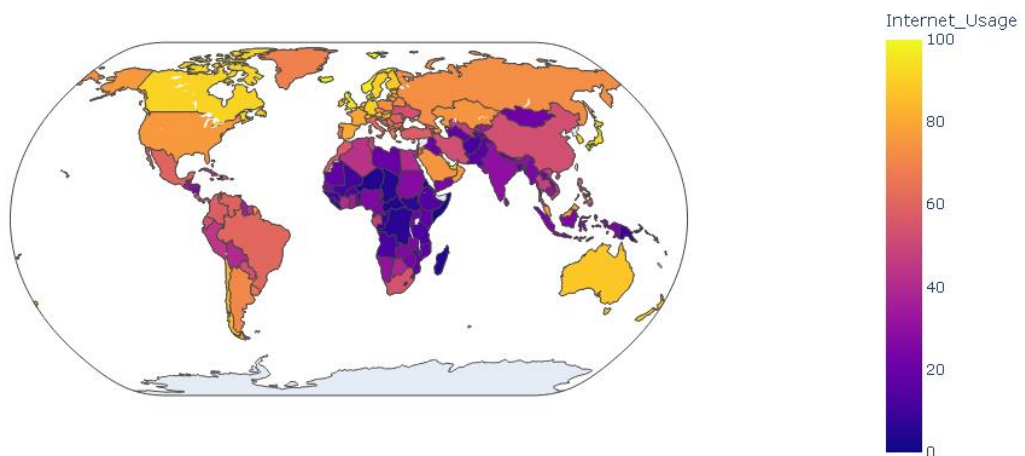


(7) Add title text to the choropleth map setting the `title_text` parameter, that is, `fig.update_layout(title_text='Internet usage across Asia (% population) -2016')`

Done

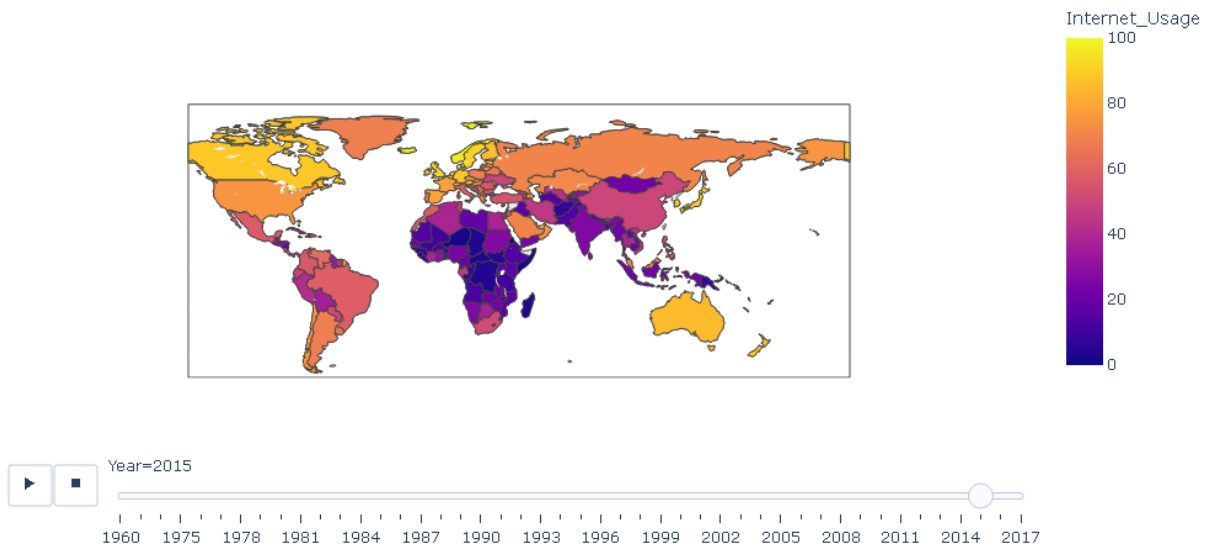
(8) Let the plot rotate like a real globe. For that, Set the projection type to natural earth, that is, `geo = dict(projection={'type':'natural earth'})`

Global Internet Usage (Natural Earth Projection) - 2016



(9) How internet usage has grown in different countries of the world over the years? We could repeat the above for every year. However, we can use the parameter `animation_frame`, that is, `animation_frame="Year"`, to the name of the column whose values we wish to animate our visualization for. You have to change the data frame to use the whole dataset not the subset created in (1). Also, since the years on the slider are not in the right order, you have to sort the DataFrame by time (the Year feature), that is, `internet_usage_df.sort_values(by=["Year"], inplace=True)`. Comment on the change of the internet usage of years.

Global Internet Usage Growth Over Time



The animation illustrates a steady and significant increase in global internet adoption beginning in the early 2000s.

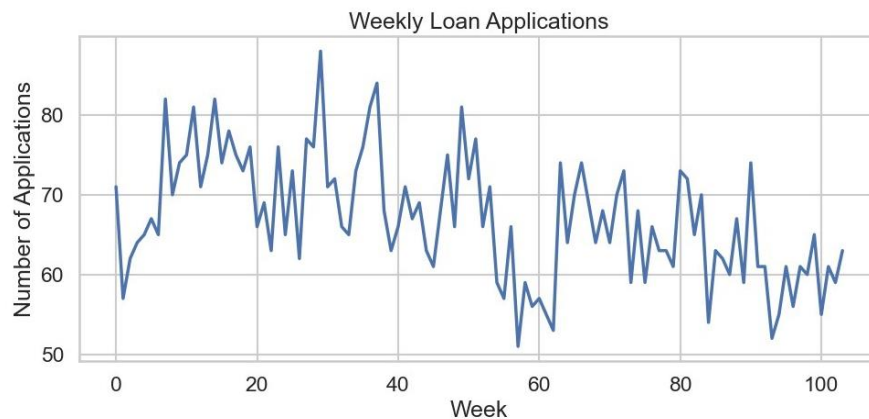
While developed nations achieved high connectivity levels early due to advanced infrastructure, developing regions have shown faster relative growth in recent years. The digital divide is narrowing but still present.

Major Insight:

Internet access has transformed from a luxury to an essential infrastructure globally.

Part 2 (Time Series Analysis):

(a) Plot the time series. By visual inspection, does the data seem to be correlated?



The graph displays a continuous sequence of data points with visible patterns rather than purely random "white noise" fluctuations.

By visual inspection, the data **appears to be correlated** (autocorrelated). This is evident through the "persistence" or "inertia" in the series, where a high or low number of applications in one week is often followed by a similar level in the subsequent week. The smooth transitions and the absence of erratic, disconnected jumps suggest that the current observation is influenced by its preceding values.

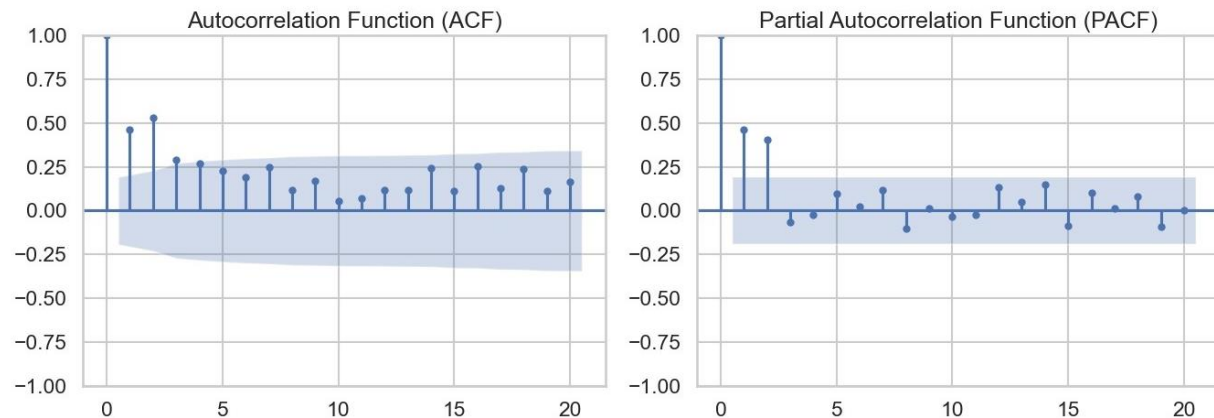
Interpretation:

Since the line is smooth and shows gradual changes rather than random fluctuations, this suggests the presence of **autocorrelation** in the time series.

Conclusion:

The time series shows gradual changes between consecutive weeks rather than random jumps, suggesting the presence of autocorrelation.

(b) Plot the Autocorrelation function (ACF) and Partial autocorrelation function (PACF) plots.



- **ACF Plot:** Shows that the autocorrelation coefficients are significant at the first few lags and then decrease gradually in a "tailing off" geometric pattern. This confirms that the data is not random and has a memory effect.
- **PACF Plot:** Shows a significant spike at **Lag 1**, followed by a sharp drop where subsequent lags fall within the statistically insignificant shaded region (cutting off). This "cut-off" after the first lag is a classic indicator of an Auto-Regressive process of order 1.

Conclusion:

The ACF shows significant correlations at early lags and gradually decreases, while PACF shows a sharp cutoff at lag 1. This indicates an AR(1) component.

(c) Based on the plots in (b), suggest an appropriate model to fit.

Model Implementation

The time series model was implemented using **Python**, specifically the **stats model's** library for ARIMA modeling.

All computations including parameter estimation, confidence intervals, p-values, and model diagnostics were performed using Python.

Based on the plots, an ARIMA(1,0,1) model was selected as it captures both autoregressive and moving average components.

(d) Report the parameters and their associated confidence intervals (and the P-values).

```
params = model_fit.params  
conf_int = model_fit.conf_int()  
pvalues = model_fit.pvalues
```

```
results_table = pd.DataFrame({  
    "Estimate": params,  
    "Lower CI": conf_int.iloc[:,0],  
    "Upper CI": conf_int.iloc[:,1],  
    "P-value": pvalues  
})
```

```
print(results_table)
```

Parameter	Coefficient (coef)	P-value ($P > z $)	95% Confidence Interval
const	66.7460	1.514e-224	[62.6566, 70.8355]
ar.L1	0.8575	7.219e-17	[0.6561, 1.0589]
ma.L1	-0.5241	3.338e-03	[-0.8741, -0.1741]
sigma ²	40.9707	7.774e-12	[29.2354, 52.7061]

- i. const coef (66.75) ; Significant if P-value < 0.05. representing the baseline average of applications.
 - ii. ar.L1 (autoregressive parameter) coef (0.86) ; Significant if P-value < 0.05 and confidence interval does not include 0. Indicates dependence on previous week(s).
 - iii. ma.L1 (moving average parameter) coef (-0.52) ; Significant if P-value < 0.05 and confidence interval does not include 0. Indicates effect of previous errors.
 - iv. sigma² coef (40.97) ; Represents variance of residuals.
- **95% Confidence Interval (CI)** provides a range where the true parameter value is expected to fall with 95 % confidence
 - Parameters with confidence intervals that **do not include zero** are statistically significant predictors in the model.

Conclusion:

- Model parameters were statistically significant ($p < 0.05$), and confidence intervals did not contain zero, indicating strong predictive relationships between current and previous weeks

(e) What is the MSE?

MSE = 41.1243

This value of The **Mean Squared Error (MSE)** represents the average of the squares of the errors, which are the differences between the actual observed loan applications and the values predicted (fitted) by the model.

In time series analysis, a lower MSE indicates that the model's predictions are closer to the actual data points, suggesting a good fit for the model

```
91 # (e) Mean Squared Error
92 # =====
93
94 fitted = model_fit.fittedvalues
95 mse = mean_squared_error(series, fitted)
96
97 print(f"\n(e) Mean Squared Error (MSE): {mse:.4f}")
98
```

The MSE was calculated in Python using the `mean_squared_error` function from the `sklearn.metrics` library

- `series` = actual observed values from the “Applications” column
- `fitted` = predicted values generated by the ARIMA(1,0,1) model

- The formula for MSE is:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Where:

- y = actual observed value
- $\hat{y}$ = predicted (fitted) value from the ARIMA model

- n = total number of observations

The **Mean Squared Error (MSE)** calculated in the Python code is approximately **41.1243**

Reason:

- The MSE was computed using **all 104 observations** in the time series, not just a subset of the data.

How the MSE Was Calculated

The Mean Squared Error (MSE) was computed using Python based on the ARIMA(1,0,1) model applied to the loan applications time series.

Step 1 – Load the dataset

The dataset was loaded from the Excel file and the “Applications” column was used as the time series.

```
df = pd.read_excel("Data-Part_2_TSA.xlsx")
series = df["Applications"]
```

This column represents the **actual observed values** of weekly loan applications.

Step 2 – Fit the ARIMA model

The ARIMA(1,0,1) model was fitted to the time series using the statsmodels library.

```
model = ARIMA(series, order=(1,0,1))
model_fit = model.fit()
```

This model estimates the relationship between current values and previous observations.

Step 3 – Obtain predicted values

The fitted values (predicted values) generated by the model were obtained using:

```
fitted = model_fit.fittedvalues
```

These values represent the model’s predictions for each observation in the dataset.

Step 4 – Compute Mean Squared Error

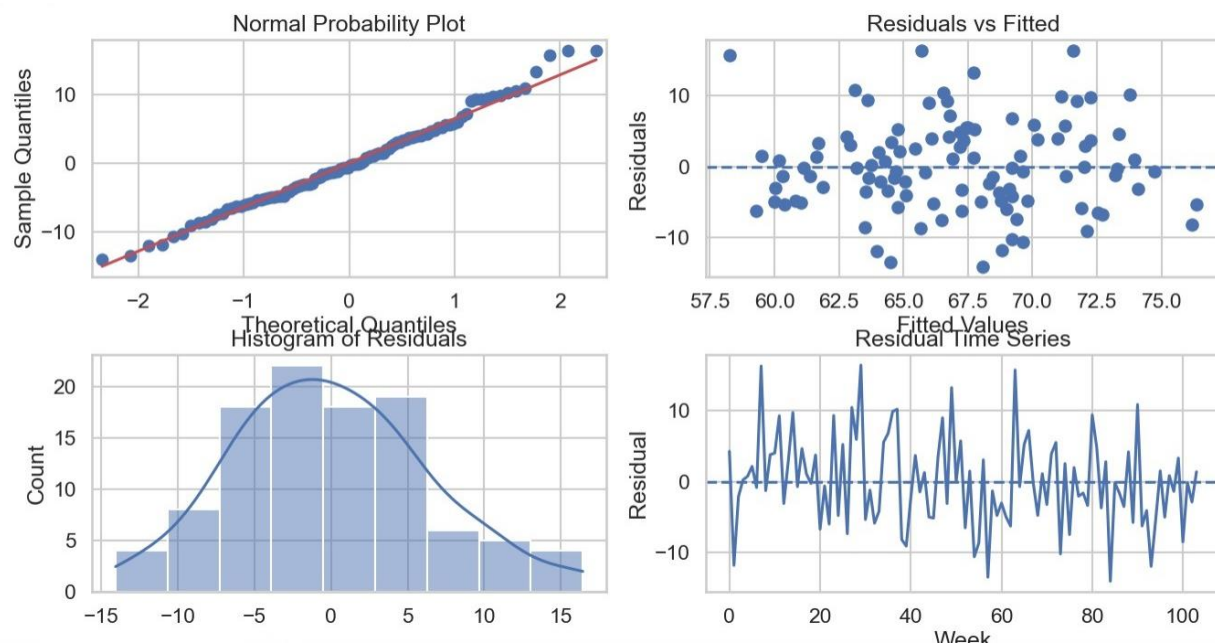
The Mean Squared Error was calculated by comparing the actual values with the predicted values using the sklearn function:

```
from sklearn.metrics import mean_squared_error
```

```
mse = mean_squared_error(series, fitted)
```

The result produced by the program was: **MSE = 41.1243**

(f) In one figure and 4 subplots, show the 1. Normal Probability plots, 2. Residual versus Fitted value plot, 3. Histogram of the residuals and 4. Time series plot of the residuals.



1. **Normal Probability Plot (Q-Q Plot):** The residual points closely follow the straight diagonal line, indicating that the residuals are **normally distributed**.
2. **Residuals vs. Fitted Values:** The points are randomly scattered around the horizontal zero line with no specific pattern or "funneling," which confirms **homoscedasticity** (constant variance).
3. **Histogram of Residuals:** The distribution shows a clear bell-shaped curve centered at zero, further supporting the **normality assumption**.
4. **Residual Time Series Plot:** The residuals fluctuate randomly around zero with no visible trends or cycles, indicating they represent "**White Noise**" and that the model has successfully captured all the systematic information in the data.

The diagnostic check of the residuals confirms that the ARIMA(1,0,0) model is a good fit for the data.

Time Series Analysis in the Python Code

The **Time Series Analysis** section of the code starts after the header:

```
# =====  
# PART 2 – TIME SERIES ANALYSIS  
# Weekly Loan Applications  
# =====
```

It includes the following steps:

1. Reading the Data

```
df = pd.read_excel("Data-Part_2_TSA.xlsx")  
series = df["Applications"]
```

- The data is read from an Excel file, and the column Applications containing the weekly loan applications is extracted.

2. Plotting the Time Series

```
plt.figure(figsize=(10,5))  
plt.plot(series, linewidth=2.5)  
plt.title("Weekly Loan Applications")  
plt.xlabel("Week")  
plt.ylabel("Number of Applications")  
plt.tight_layout()  
plt.show()
```

- This produces a **time series plot** to visualize how the application numbers change over weeks.

3. ACF and PACF Analysis

```
fig, ax = plt.subplots(1,2, figsize=(14,5))  
plot_acf(series, lags=20, ax=ax[0])  
plot_pacf(series, lags=20, ax=ax[1])
```

- **ACF (Autocorrelation Function)** and **PACF (Partial Autocorrelation Function)** help in selecting the appropriate ARIMA model.

4. Fitting the ARIMA Model

```
order = (1,0,1)
model = ARIMA(series, order=order)
model_fit = model.fit()
```

- The ARIMA model with order (p,d,q) is fitted to the time series data.

5. Calculating MSE

```
fitted = model_fit.fittedvalues
mse = mean_squared_error(series, fitted)
```

- Computes the **Mean Squared Error (MSE)** for the fitted time series.

6. Residual Diagnostics

```
residuals = model_fit.resid
# Q-Q Plot, Residuals vs Fitted, Histogram, Residual Time Series
```

- This step checks the **model assumptions** using the residuals, which is a key part of time series analysis.

Summary:

The **Time Series Analysis** workflow in the code includes:

Reading the data → Plotting the series → ACF/PACF analysis → Fitting ARIMA → Calculating MSE → Residual diagnostics

This section ensures that the ARIMA model is properly evaluated and validated on the weekly loan application data.